

Universal Authentication Concerns: Real World Problems and Production Approaches

This document covers the twelve concerns that every authentication implementation must address, regardless of whether you are using session cookies, JWTs, or OAuth/OIDC. For each topic: the real world problems, how each mechanism handles it, the tradeoffs, and what production systems actually do.

1. Logout

The real problems

Logout sounds trivial until you actually try to make it secure. The core tension is this: with stateless tokens, "logging out" on the client does not make the token invalid on the server. If an attacker captured that token before logout, they can keep using it until it naturally expires, even though the legitimate user has logged out.

Other real scenarios: an admin needs to force-logout a compromised account immediately, not wait for token expiry. A user logs out on their phone but their laptop session stays active. A company offboards an employee and needs every one of their sessions killed instantly, across every device and every app that uses SSO.

How each mechanism handles it

Session-based: Trivial and immediate. The server deletes the session record from its store (Redis, DB). The cookie the browser holds becomes a pointer to nothing. The very next request fails authentication. This is the gold standard for "logout actually means logout."

JWT (access + refresh): The access token cannot be un-issued. It remains cryptographically valid until its `(exp)` claim passes, no matter what the server does. What you CAN do is revoke the refresh token (delete it from the server's token table), which prevents the user from getting a NEW access token once the current one expires. So logout is "soft": the user is logged out for all future token refreshes, but the current access token keeps working for up to its remaining lifetime (which is why access tokens are kept short, 5 to 15 minutes).

For true immediate revocation, you need a deny-list (a Redis set of revoked token IDs checked on every request). This reintroduces a server-side lookup, which is the exact thing JWTs were chosen to avoid, so it is usually reserved for high-severity cases (admin-forced logout, detected compromise) rather than every logout.

OAuth/OIDC: Same soft-logout problem as JWT for the access token, plus an additional layer. The provider has a token revocation endpoint (`(/revoke)`) your app should call on logout. For SSO scenarios, OIDC defines an "end session endpoint" (RP-initiated logout) that can propagate logout across every application that shares that identity provider

session, this is what lets "log out of Google" sign you out of Gmail, YouTube, and Google Docs simultaneously.

Pros and cons

Session-based wins decisively on logout correctness: instant, simple, no edge cases. Its cost is that every request needs a server-side store lookup, which becomes a scaling concern at very high request volumes (though Redis handles this fine for the vast majority of applications, this is rarely the actual bottleneck people think it is).

JWT without a deny-list is fast and stateless but has a "logout latency" equal to the access token's lifetime. This is an acceptable tradeoff for most applications because a 10-15 minute exposure window after logout is a low-severity risk for typical apps. JWT with a deny-list gets you immediate revocation but you have re-added the server-side state you were trying to avoid, so at that point session-based architecture is often simpler to reason about.

Production-ready approach

For the vast majority of apps: short-lived access tokens (10-15 minutes) plus refresh token revocation on logout. This covers normal logout completely and limits the worst-case exposure to a small window.

For anything with an admin "force logout user" or "kill this session" feature, you need a server-side deny-list or a session-table approach regardless of whether you call your access credential a JWT or a session ID, the architecture converges.

Does the site type change this?

Yes, significantly. A banking app or admin console needs the deny-list approach or pure sessions, the 10-minute exposure window is unacceptable when the action could be a wire transfer. A content site, blog, or marketing SaaS dashboard is fine with the soft-logout JWT approach, the worst case is someone reads an article they should not have. An enterprise SaaS with "remove this employee's access immediately" as a sales requirement needs server-tracked refresh tokens (a session table) no matter what you call the access credential.

2. Session/Token Expiry Handling

The real problems

A user is filling out a long form, browsing a dashboard with multiple widgets, or in the middle of a multi-step checkout when their token expires. If handled badly: their next action throws a confusing error, or worse, multiple simultaneous API calls (a dashboard loading five widgets) all get 401 at once and each independently tries to refresh the token, causing a "thundering herd" of refresh requests that can trigger rate limits or, in rotating-refresh-token systems, false-positive theft detection (discussed in section 5).

Another problem: after a refresh succeeds, the original failed requests need to be retried with the new token, not just discarded, otherwise the user's action silently fails.

How each mechanism handles it

Session-based: Often uses sliding expiration, every request that hits the server extends the session's TTL. So as long as the user is active, the session effectively never expires. If it does expire (user truly inactive past the absolute max), the next request simply fails and the user is redirected to login, there is no "refresh" concept because there is nothing to refresh, the session either exists or it does not.

JWT with refresh token: The access token has a fixed lifetime. The client must detect expiry (either proactively via reading the `exp` claim, or reactively via a 401 response) and call the refresh endpoint to get a new access token. This requires meaningful client-side engineering: a request interceptor, a mutex/lock so concurrent 401s share one refresh call, and a retry queue for the failed requests.

OAuth/OIDC: Identical pattern to JWT, the access token from the provider expires, the refresh token (if the provider issued one and your scopes include `offline_access`) is exchanged at the provider's token endpoint for a new access token.

Pros and cons

Sliding session expiration is very user-friendly (active users are never logged out) but has a security tradeoff: an attacker who has stolen a session cookie and keeps making requests with it keeps the session alive indefinitely, there is no natural "this credential is now stale" boundary unless you also enforce an absolute maximum lifetime alongside the sliding window.

JWT refresh requires more client-side complexity but decouples "how long can this credential be used without re-authentication" (refresh token lifetime, can be long) from "how long is any single credential valid if stolen" (access token lifetime, short). This is generally considered the better security model for anything beyond simple session-based sites, at the cost of needing solid interceptor/queue logic.

Production-ready approach

Implement a single shared refresh promise: the first 401 (or proactive expiry detection) triggers one refresh call, and any other requests that hit 401 while that refresh is in-flight await the same promise rather than firing their own. Once refresh succeeds, replay the queued failed requests with the new token. If refresh fails (refresh token also expired or revoked), clear auth state and redirect to login, preserving the current URL so the user returns to where they were.

Does the site type change this?

Real-time and long-session apps (collaborative editors, dashboards left open all day, trading platforms) need this to be airtight, users genuinely keep tabs open for hours and any friction here is highly visible. Simple content sites or apps where sessions are naturally

short (a single checkout flow) can get away with a much simpler "expired? redirect to login" approach without sophisticated queuing.

3. Idle Timeout

The real problems

This is a different concern from token expiry: the credential might still be cryptographically valid, but the user walked away from their desk 20 minutes ago and left the app open on a shared or unlocked computer. Compliance frameworks (PCI DSS, HIPAA, many banking regulations) often mandate automatic logout after a fixed period of inactivity, commonly 15 minutes, regardless of how long the underlying token would otherwise last.

The implementation challenges: detecting "activity" reliably (mouse movement, key presses, clicks, but not just any network request, since background polling could falsely reset the timer), warning the user before logging them out (so they do not lose unsaved work), and syncing the idle timer across multiple open tabs (activity in tab 1 should reset the timer for tabs 2 and 3 too).

How each mechanism handles it

Session-based: Can be enforced server-side by storing a `lastActivity` timestamp on the session record and checking it on every request, independent of anything the client does. This is robust because it cannot be bypassed by a user (or attacker) tampering with client-side JavaScript.

JWT: Idle timeout is fundamentally a client-side concern for pure stateless JWTs, since the server has no record of "when did this token last get used." A short access token lifetime helps somewhat (an idle user's token will simply expire naturally), but a true 15-minute compliance-grade idle timeout enforced server-side requires the server to track last-used time per token, which again reintroduces server-side state (similar to the deny-list discussion in section 1).

Pros and cons

Client-only idle timeout (listening for `mousemove`/`keydown`/`click`, resetting a timer, calling logout when it fires) is easy to implement and fine for UX-driven idle warnings ("are you still there?"). But it is trivially bypassable, a user (or malicious script) can simply prevent that JS from running, so it cannot be relied upon for actual compliance.

Server-enforced idle timeout (session-based, or JWT with a tracked last-used timestamp) is robust and audit-friendly but adds the server-side lookup cost on every request.

Production-ready approach

For compliance-driven idle timeout: enforce it server-side (session-based architecture, or a tracked-token table for JWTs), this is non-negotiable for an audit. Layer a client-side idle timer on top purely for UX, showing a "you will be logged out in 30 seconds, click to stay

logged in" modal before the server-side timeout actually fires, and use `BroadcastChannel` so activity in any tab resets the timer for all tabs.

Does the site type change this?

Banking, healthcare, and anything handling PII/PHI under regulation: mandatory, server-enforced, typically 10-15 minutes, non-configurable by the user. Enterprise B2B SaaS: often configurable per-organization by the admin (a security setting tenants can tune).

Consumer apps (social media, streaming, e-commerce browsing): almost never implemented, sessions are designed to be long-lived for convenience.

4. Token Storage (Where to Keep Credentials in the Browser)

The real problems

This is one of the most consequential decisions in frontend auth architecture because it determines your blast radius if an XSS vulnerability is ever found anywhere in your app (including in a third-party script or npm dependency you did not write).

The scenario that drives this: your app includes a third-party analytics script, or a user-generated content area, and it gets compromised (supply chain attack or unsanitized input). If your auth token sits in `localStorage`, that injected script can read it with one line of code and send it to an attacker's server, who can now impersonate the user indefinitely (or until the token expires) from anywhere, no further access to the victim's browser needed.

How each mechanism handles it

Session ID (cookie-based): Conventionally stored in an `httpOnly` cookie, meaning JavaScript cannot read it at all (`document.cookie` will not show it). An XSS attacker running script in the page can still make authenticated requests (the browser attaches the cookie automatically) but cannot exfiltrate the cookie value for later or offline use. This is a meaningfully smaller blast radius.

JWT access token: Three common locations, each with different tradeoffs:

- In-memory (a JS variable / React state): not persisted, lost on page refresh, but not sitting in any browser storage an attacker can dump wholesale. Requires re-establishing it on every page load via a refresh token.
- `localStorage`: persists across reloads, very convenient, but fully readable by any script running on the page, the worst option from an XSS perspective.
- `sessionStorage`: similar risk profile to `localStorage` but cleared when the tab closes.

JWT refresh token: Best practice is an `httpOnly`, `Secure`, `SameSite=Strict` (or `Lax`) cookie, for the same reason as session cookies, it is the longest-lived and most valuable credential, so it deserves the strongest protection.

`localStorage` is the easiest to implement (just `localStorage.setItem`) and works naturally for cross-origin API calls without cookie/CORS complications, which is why it remains common despite the XSS risk. `httpOnly` cookies are far safer against XSS but introduce CSRF concerns (section 6) and cross-origin cookie complexity (`SameSite`, `credentials: 'include'`, CORS configuration) when your frontend and API are on different domains.

In-memory storage for the access token gives you the best of both, nothing persistent to steal, but you need a working silent-refresh flow (section 12) to re-acquire an access token on every page load, adding complexity.

Production-ready approach

The increasingly recommended pattern, especially promoted by identity providers like Auth0 and Okta, is the **Backend-for-Frontend (BFF) pattern**: the SPA never receives or stores any access/refresh tokens at all. The browser holds only an `httpOnly` session cookie to your OWN backend. Your backend holds the real tokens (JWT, OAuth tokens, whatever) server-side and proxies API calls on the user's behalf. This combines the XSS-resistance of cookie-based sessions with the architectural benefits of token-based backends, and sidesteps the localStorage-vs-cookie debate entirely by keeping tokens out of the browser.

If a BFF is not feasible: access token in memory, refresh token in an `httpOnly` `SameSite=Strict` cookie, is the next best widely-used pattern.

Does the site type change this?

Native mobile apps have a different threat model entirely, they use the OS keychain/keystore, which is sandboxed per-app and not exposed to a browser-style XSS vector, so the localStorage-vs-cookie debate does not directly apply there. High-value targets (fintech, crypto, healthcare) should strongly prefer the BFF pattern or `httpOnly` cookies. Low-stakes internal tools with no third-party scripts and tightly controlled dependencies often run localStorage-based JWTs in production without major incident, simply because their actual XSS attack surface is small, though this is a risk-acceptance decision, not a best practice.

5. Token Rotation

The real problems

The scenario that motivates rotation: a refresh token is long-lived (days to weeks) by design. If it is ever stolen (via a compromised device, a logging mistake, a malicious browser extension), the attacker can mint fresh access tokens indefinitely until the refresh token's long expiry finally arrives, or forever if it never expires.

Token rotation addresses this by making refresh tokens single-use: each time a refresh token is used, the server issues a brand new refresh token and immediately invalidates the old one. This creates a powerful side effect: if the OLD (now-invalid) refresh token is ever used again, that is a strong signal someone has a copy of a token that the legitimate client already rotated away from, meaning theft has occurred.

The tricky real-world problem: network retries. If a client's refresh request times out but actually succeeded server-side, the client might retry with the now-rotated-away token, triggering a false "theft detected" and locking out a legitimate user. This is a genuinely hard distributed-systems problem.

How each mechanism handles it

Session-based: Rotation in the classic sense is less relevant since the session ID is validated against server state on every request anyway (there is nothing "stale" to detect). However, a related practice, regenerating the session ID after login or after a privilege change (e.g., password change, role upgrade), is important to prevent session fixation attacks, where an attacker tricks a victim into using a session ID the attacker already knows.

JWT refresh tokens: This is where rotation matters most. On every refresh, issue a new refresh token (and access token), store the new one, invalidate the old one. Reuse of an invalidated token triggers the theft response.

API keys: Rotation is manual and operational, not automatic. Most platforms (AWS, Stripe, GitHub) support generating a new key while the old remains valid for a grace period, letting you update configuration without downtime, then revoking the old key once migration is confirmed. There is no automatic "reuse detection" because API keys are not single-use by design.

Pros and cons

Aggressive theft response (revoke the entire "token family", forcing re-login on every device) is the most secure reaction but can be a poor user experience if triggered by a false positive from a network glitch, a user could find themselves logged out of all five of their devices because one mobile network retry raced. A grace window (allow the "old" token to be used once more within a few seconds, treating it as a likely retry rather than theft) reduces false positives but slightly weakens the theft-detection guarantee.

Production-ready approach

Implement rotation with reuse detection, but add a short grace period (a few seconds) where a just-rotated-away token is tolerated as a probable retry rather than immediately triggering family-wide revocation. On confirmed reuse outside the grace window, revoke the token family and proactively notify the user ("we detected a new sign-in, if this wasn't you, secure your account").

Does the site type change this?

High-security platforms (crypto exchanges, banking, anything with stored payment

methods) should lean toward the aggressive end, full family revocation plus mandatory user notification on any reuse signal, even at some UX cost. Typical SaaS and consumer apps benefit from rotation with a generous grace window, prioritizing not locking out legitimate users over flaky connections.

6. CSRF (Cross-Site Request Forgery) Protection

The real problems

The classic scenario: a user is logged into their bank (cookie-based session) in one tab. In another tab, they visit a malicious site that contains a hidden auto-submitting form pointed at `(https://bank.com/transfer?to=attacker&amount=10000)`. The browser, following normal cookie behavior, attaches the bank's session cookie to that request automatically. The bank's server sees a request with a valid session cookie and processes the transfer, it has no way to know the request did not originate from the bank's own page.

This attack is specific to **cookie-based** authentication, because cookies are attached by the browser automatically to any request to that domain, regardless of which site initiated the request.

How each mechanism handles it

Session-based (cookie auth): Vulnerable by default, must be explicitly protected. Two main defenses: the `(SameSite)` cookie attribute (`(Strict)` prevents the cookie being sent on any cross-site request including this one; `(Lax)` allows it for top-level navigation/GET but blocks it for the cross-site POST in the example above, covering most CSRF cases while still allowing things like clicking a link from an email to take you to a logged-in page). The second defense is the synchronizer token pattern: the server embeds a random CSRF token in the page, the client must include it in a custom header or form field on every state-changing request, and the malicious site cannot read or replicate that token (same-origin policy prevents it from reading the page's content).

JWT in Authorization header: Largely immune to CSRF by construction. A malicious site's auto-submitting form cannot set a custom `(Authorization: Bearer ...)` header, only the browser's automatic cookie-attachment creates the CSRF problem, and header-based tokens are never attached automatically. This is one of the genuine security advantages of header-based auth over cookies, though it comes paired with the XSS tradeoff discussed in section 4 (a token a script CAN read for a header is also a token a script CAN steal).

OAuth/OIDC: Has its own, different CSRF concern in the authorization redirect flow itself, an attacker could initiate an OAuth flow and trick a victim into completing it, binding the victim's account to the attacker's third-party identity. This is mitigated by the `(state)` parameter, a random value your app generates before redirecting, and verifies matches on the callback.

Pros and cons

`SameSite=Strict` is the strongest cookie defense but breaks legitimate cross-site scenarios, for example, a payment provider redirecting the user back to your site after checkout would not carry your auth cookie if it is `Strict`, since that "return" navigation is itself cross-site. `SameSite=Lax` is the commonly recommended default, it blocks the dangerous cross-site POST scenario while allowing normal top-level navigation cookies to flow.

Header-based JWTs sidestep CSRF entirely but, as covered in section 4, shift more of the risk burden onto XSS defense.

Production-ready approach

For cookie-based auth (including the BFF pattern from section 4): set `SameSite=Lax` (or `Strict` if your flows allow it) as the baseline, and add the synchronizer token pattern (or the simpler "double-submit cookie" variant, where a CSRF token is set as a non-httpOnly cookie AND must be echoed back as a custom header, the server compares the two, an attacker site cannot read the cookie to replicate the header due to same-origin policy) for all mutating endpoints. Also verify the `Origin`/`Referer` header server-side as a defense-in-depth layer.

For header-based JWT auth, CSRF is largely a non-issue, focus your defenses on XSS instead.

Does the site type change this?

Anything with state-changing actions that have real-world consequences (banking transfers, e-commerce purchases, account settings changes, admin actions) must have CSRF protection regardless of the broader auth architecture, this is one of the few items on this list that is essentially non-negotiable for any cookie-based system. Read-only public content sites with cookie-based sessions have lower stakes but should still set `SameSite=Lax` as a basic hygiene measure, it costs nothing.

7. Concurrent Sessions / Multi-Device Session Management

The real problems

Real product requirements that fall under this: a "manage your devices" page (Netflix, Google, banking apps all have this), an enterprise admin needing to remotely revoke a specific employee's laptop session after they report it stolen, a security alert ("new login from an unrecognized device, was this you? if not, click to log it out"), and device limits (a streaming plan that only allows 4 concurrent devices).

All of these require the same underlying capability: the server must be able to enumerate, identify, and individually revoke specific active credentials, tied to metadata like device type, IP address, location, and last-active time.

How each mechanism handles it

Session-based: Naturally supports this. Each session is a row in a database table. List all rows for a user, show them as "devices" (storing user-agent and IP at creation time), delete a specific row to revoke that device.

Pure stateless JWT: Cannot support this at all. If the server keeps no record of issued tokens, there is no list to show and nothing to individually revoke, a token is valid until it expires, full stop, for every token, with no per-token distinction possible.

JWT with tracked refresh tokens (the realistic hybrid): The access token stays stateless and short-lived for performance, but the server maintains a table of issued refresh tokens, one row per device/session, with metadata. This gives you the management UI capability (the refresh token table IS your sessions table) while keeping the high-frequency access-token validation fast and stateless.

OAuth/OIDC: Identity providers typically expose a "connected apps and devices" page backed by the same kind of grant/session tracking on their end.

Pros and cons

This is one of the clearest examples of "pure stateless JWT" being a theoretical ideal that almost no real production system actually uses unmodified, the moment a product requirement like "let me kick a device" appears (and it almost always does, even if not in version 1), you need server-side tracking somewhere. The good news is that tracking only the refresh tokens (which are issued far less frequently than access tokens) keeps the server-side lookup load manageable, you are not adding a DB hit to every API call, only to refresh calls and to the device-management page itself.

Production-ready approach

Maintain a sessions/refresh-token table keyed by user, with columns for device info (parsed user-agent), IP, creation time, and last-used time. Expose a `/sessions` endpoint the frontend uses for the "active devices" page, and a `DELETE /sessions/:id` endpoint for per-device revocation. This table is your single source of truth regardless of whether the access credential itself is a JWT or an opaque session ID.

Does the site type change this?

Enterprise B2B SaaS buyers frequently ask about this in security questionnaires, "can an admin revoke a specific user's active sessions" is close to a checkbox requirement for selling into security-conscious organizations. Consumer streaming and subscription services use this for device-limit enforcement, a core business requirement. Small internal tools or single-user apps often skip this entirely, there is rarely a need to manage "devices" for a tool only one person uses.

8. Cross-Tab Synchronization

The real problems

A user has your app open in five browser tabs. They click "log out" in tab 3. Tabs 1, 2, 4, and 5 have no idea this happened, they continue to show the authenticated UI, and the next time the user clicks something in one of them, it either silently fails (401) or, worse, briefly shows stale data before erroring confusingly.

A related scenario: the user logs into a DIFFERENT account in one tab (account switching, common in tools that support multiple workspaces), the other tabs are now showing data for the wrong account until refreshed.

This concern is entirely orthogonal to which auth mechanism you use, session, JWT, or OAuth, all of them face this if the app is a SPA that users commonly run in multiple tabs, because the problem lives in client-side application state, not in the credential itself.

How each mechanism handles it (and why this is mostly the same across mechanisms)

For cookie-based auth, there is a partial natural mitigation: cookies are shared across all tabs of the same origin automatically, so once the cookie is cleared (logout), the NEXT network request from any tab will fail authentication on its own. However, the in-memory React/Vue state in each tab (which says "user is logged in, here is their name") does not know this until that failed request happens, so the UI can still look authenticated for a while.

For JWT stored in memory or localStorage, there is no automatic cross-tab sharing of in-memory state at all, and even localStorage requires an explicit listener to react to changes made by other tabs.

The actual solution (same regardless of mechanism)

The `storage` event fires in OTHER tabs (never the tab that made the change) whenever a `localStorage` key is modified, so even apps that do not store tokens in localStorage can use a dedicated "auth-event" key purely as a cross-tab signal. The cleaner modern API is `BroadcastChannel`, a same-origin publish/subscribe channel: on logout, the tab that initiated it does `channel.postMessage({type: 'LOGOUT'})`, and every other tab's listener clears its local auth state and redirects to login.

Pros and cons

`localStorage` events work everywhere but are a slight repurposing of a storage mechanism for messaging, feels a bit hacky. `BroadcastChannel` is purpose-built and cleaner but has marginally less universal browser support (a non-issue for any modern target). Both are lightweight and require no server changes, this is purely a client-side application concern.

Production-ready approach

Use `BroadcastChannel` (with a `localStorage`-event fallback if you need to support very old browsers) for auth-state changes: logout, login, and token refresh in one tab broadcast

to all others, which update their local auth context accordingly. Combine this with the idle-timeout cross-tab reset mentioned in section 3, the same channel can carry both message types.

Does the site type change this?

Heavy multi-tab usage patterns, email clients, project management tools, admin dashboards, anything people tend to keep open in several tabs throughout a workday, make this important for a polished experience. Single-flow apps like a one-page checkout or a landing-page signup form rarely have this problem in practice, since users are unlikely to have multiple tabs of the same transient flow open.

9. Permission Checks / Role-Based Access Control on the Frontend

The real problems

The canonical failure mode: an admin panel shows a "Delete User" button only when `user.role === 'admin'`. A regular (non-admin) user opens browser dev tools, finds the network request that button triggers (`DELETE /api/users/42`), and replays it directly using their own valid (non-admin) token. If the backend does not independently check the role on that endpoint, and only the frontend hid the button, the delete succeeds. This is "broken access control," and it has been at or near the top of the OWASP API Security Top 10 for years because it is so common and so easy to get wrong.

The deeper truth: authentication (proving who you are) and authorization (checking what you are allowed to do) are completely separate concerns, and the auth MECHANISM (session/JWT/OAuth) only solves the first one. It tells the server "this request is from user 42." Whether user 42 is allowed to delete other users is a business-logic decision that must be checked on the server, every single time, regardless of what the token says or what the frontend rendered.

How each mechanism handles the "what are my permissions" question on the client

Session-based: Typically the client does not know the user's roles/permissions just from the session cookie (it is an opaque ID). The frontend calls a `/me` endpoint after login to fetch the current user's roles, this is always fresh because it reflects the database at the moment of the call.

JWT: Roles/permissions are often embedded directly as claims in the token (`"roles": ["editor", "billing_admin"]`), avoiding an extra API call, the frontend can read this from the decoded token immediately. The downside: if an admin revokes a role mid-session, the user's existing JWT still contains the OLD roles until that token expires (or until refresh, if the refresh flow re-fetches current roles from the database, which it should).

OAuth/OIDC: Scopes in the access token define what the token is allowed to do at a coarse level (e.g., `read:contacts`), but fine-grained application-level roles are usually still a separate concern fetched from your own backend.

Pros and cons

Embedding roles in JWT claims is faster (no extra round trip) but introduces "permission drift," a window where the client's view of the user's permissions is stale relative to the database. Session-based or always-fetched permissions are guaranteed fresh but cost a request.

Production-ready approach

On the client: use cached permission data (from JWT claims or a `/me` call) purely for UX, conditionally render buttons, gray out disabled actions, hide entire navigation sections a user cannot access. This makes the UI feel coherent and prevents users from even attempting things they cannot do.

On the server: every single mutating endpoint independently re-validates the requesting user's permission against the current database state for that specific action and resource, never trusting the token's embedded role claims as the final word for sensitive operations (at minimum, for high-stakes actions like deletion, payment, or permission changes, re-check against the DB even if the JWT claims look sufficient).

Does the site type change this?

Multi-tenant B2B SaaS with hierarchical roles (organization owner, admin, member, guest, often PER-WORKSPACE rather than global) usually outgrows simple role-based checks and needs attribute-based access control (ABAC), rules like "can edit if you are the owner, OR you are an editor AND you were specifically assigned to this resource." This kind of nuance cannot reasonably live in a JWT claim, it needs a server-side policy layer (tools like Open Policy Agent or Casbin are common here). Simple apps with 2-3 global roles (user/admin) are fine with straightforward role checks embedded in tokens or fetched once at login.

10. XSS (Cross-Site Scripting) Defense

The real problems

A blog or forum allows comments. The comment sanitization has a gap, an attacker posts a comment containing a `<script>` tag that reads `localStorage.getItem('authToken')` and sends it to an attacker-controlled server. Every visitor who views that comment (including logged-in moderators and admins) has their token silently exfiltrated, and the attacker can now act as them, potentially for as long as that token remains valid.

A second, increasingly common vector: a third-party script you depend on (an analytics tag, an ad network, a compromised npm package in your build) gets compromised at the source, injecting malicious code into every page of your app without any "user input" being involved at all, this is a supply-chain XSS attack and has affected major companies.

How each mechanism's storage choice affects the blast radius

This connects directly back to section 4. If the access token sits in `localStorage`, an XSS payload can both USE it (make authenticated API calls as the victim, live) AND STEAL it (exfiltrate the raw value for offline/later replay from the attacker's own machine, valid until expiry). This is the worst case.

If the credential is an `httpOnly` cookie (session-based, or a JWT refresh token stored this way), an XSS payload running in the page can still USE it (the browser attaches the cookie automatically to requests the malicious script makes), but cannot STEAL the raw value for replay from elsewhere, because `document.cookie` will not expose an `httpOnly` cookie. This significantly limits the attack to "while the victim has the page open" rather than "indefinitely, from anywhere."

In-memory storage is somewhere in between, not trivially dumpable via a storage API, but a sufficiently targeted script running in the same execution context could still potentially access in-scope JS variables, so it is not a complete shield either, it mainly raises the bar.

Pros and cons of defenses

Input sanitization (escaping HTML in user-generated content, using libraries like DOMPurify for any case where raw HTML rendering is genuinely needed) addresses the most common injection vector but does nothing against supply-chain attacks via trusted third-party code. Content-Security-Policy headers restrict which script sources the browser will execute at all (`script-src 'self' https://trusted-cdn.com`), which helps against both injected `<script>` tags AND unauthorized third-party scripts, but can be a significant effort to configure correctly for apps with many third-party integrations, and breaks inline scripts/styles unless carefully allow-listed.

Production-ready approach

Layered defense: React (and most modern frameworks) escape text content by default, the main danger is `dangerouslySetInnerHTML` (React) or `v-html` (Vue) and similar APIs, audit every use of these and run user content through DOMPurify if raw HTML rendering is required. Set a Content-Security-Policy header restricting script sources to your own origin and an explicit allowlist of trusted third parties. Use Subresource Integrity (SRI) hashes for any scripts loaded from a CDN, so a compromised CDN file fails to execute rather than silently running. Keep the most sensitive credential (refresh token, or the only token in a BFF setup) in an `httpOnly` cookie so that even a successful XSS has a limited blast radius (session-duration, not indefinite). Run regular dependency audits (`npm audit` and equivalents) given how common supply-chain compromises have become.

Does the site type change this?

Sites with significant user-generated content (forums, comment sections, wikis, anything rendering user-submitted markdown or rich text) have the highest direct injection risk and need the strictest sanitization and CSP. Even "internal tool, no UGC" apps are not exempt,

the supply-chain vector via third-party scripts and dependencies applies to literally every app with an npm-based build, which is effectively all of them.

11. Protected Routes

The real problems

A user bookmarks a deep link, say `/dashboard/reports/42`. Later, not logged in (session expired, or first visit from the bookmark), they open that URL directly. A naive implementation either crashes, shows a broken page, or redirects to a generic `/login` and then, after successful login, dumps the user on the homepage, losing the context of where they were actually trying to go, a frustrating experience if it happens repeatedly.

A second, more subtle problem: the "flash of unauthenticated content." On page load, the app does not yet know if the user is logged in (it needs to check a cookie's validity with the server, or decode and verify a token). If the route guard defaults to "not authenticated" while this check is in progress, the user briefly sees the login page flash before flipping to the dashboard once the check resolves, jarring and unprofessional-looking.

How each mechanism affects the auth check itself

Session-based: The client genuinely does not know if the session is valid just by looking at the cookie (it cannot read an `httpOnly` cookie, and even if it could, the cookie's mere presence does not mean the server-side session is still valid). It must make a network call (typically to `/me` or similar) to find out, this means there is ALWAYS a brief "loading" period on app start, by necessity.

JWT: If the access token is available client-side (not in the BFF pattern), the client CAN do a quick local check, decode the token (no verification needed for this UX purpose) and check if `(exp)` has passed. This gives an instant "definitely not logged in" signal without a network round trip if the token is clearly expired. However, "not expired locally" does not guarantee server-side validity (the token could have been revoked), so for anything sensitive, a background verification call is still warranted, the local check is for instant UX, not security.

Pros and cons

The JWT local-decode approach gives snappier perceived performance (no flash of login page for users who are clearly logged in, since the check is synchronous and local), at the cost of needing a background verification for true security-sensitive determinations. The session-based "always ask the server" approach is simpler conceptually (one source of truth) but means every app load has an unavoidable network round trip before the UI can confidently render either state.

Production-ready approach

Model auth state as three distinct values, not a boolean: `(loading)`, `(authenticated)`,

`unauthenticated`). Route guards render a neutral loading state (skeleton screen or spinner, never the login page) while in `loading`. Once resolved, render the appropriate page. On redirect-to-login due to an unauthenticated access attempt, pass the originally-requested path along (`/login?redirect=/dashboard/reports/42`), and after successful login, the router pushes the user to that saved path instead of a default homepage.

Does the site type change this?

Apps with meaningfully deep-linkable content, documentation sites with login-gated pages, dashboards with many sub-pages, B2B tools where users share specific links to colleagues, benefit significantly from the redirect-back behavior, it is the difference between "click link, log in, arrive exactly where intended" and "click link, log in, now navigate manually to find what you wanted again." Apps where essentially the entire experience is behind a single login wall with little deep-linking (many consumer mobile-first apps) can get away with a simpler "not logged in -> always show login -> always land on home" flow without much loss.

12. Silent Refresh

The real problems

A user is 18 minutes into filling out a long, detailed form (an insurance application, a multi-step job application, a tax form) when their 15-minute access token expires. If this is not handled gracefully, their final "Submit" click returns a 401, and depending on how the form is built, they may lose everything they typed, a catastrophic UX failure that directly costs conversions.

A second problem: a dashboard loads and fires off requests to ten different API endpoints simultaneously for ten widgets. If the access token happens to be expired at that moment, all ten requests get 401 at roughly the same instant. A naive implementation has each of the ten requests independently trigger its own token-refresh call, resulting in ten simultaneous refresh requests. Beyond being wasteful, if your refresh tokens are rotating and single-use (section 5), this can trigger a false "reuse detected, possible theft" response and lock the user out entirely, a self-inflicted denial of service caused purely by bad client-side coordination.

How each mechanism handles this (and where it does/doesn't apply)

JWT and OAuth with refresh tokens: This is THE textbook scenario refresh tokens exist for. The client either proactively refreshes before expiry (decode the access token's `exp` claim, set a timer to refresh roughly 60 seconds before that time) or reactively refreshes on the first 401 it encounters.

Session-based: The "silent refresh" problem mostly does not arise in the same form, because sessions are typically sliding-expiration, every authenticated request the user makes during their form-filling extends the session automatically, so by the time they click

Submit, the session has been continuously renewed by their own activity. If a session-based system DOES have a hard absolute expiry that can be hit mid-task, there is generally no refresh mechanism, the user simply has to log in again, sessions do not have a refresh-token equivalent in the typical implementation.

Pros and cons

Proactive (timer-based) refresh means the user should never actually SEE a 401 in normal operation, smoother, but requires correctly decoding tokens client-side and managing timers reliably, including handling browser tab visibility changes (background tabs are often throttled by the browser, so a `setTimeout` for "refresh in 14 minutes" might not fire exactly on time if the tab was backgrounded).

Reactive (refresh-on-401) is simpler to implement (it is just an interceptor) but guarantees that at least one request fails first before the retry succeeds, slightly less smooth, and absolutely requires the request-queuing/mutex logic from section 2 to avoid the thundering-herd problem described above.

Production-ready approach

Use both, layered: a proactive timer as the primary mechanism (refresh ~60 seconds before known expiry, recalculating on tab focus to account for background throttling), with a reactive 401-interceptor as a safety net for clock skew or edge cases the timer missed. Critically, implement a single shared "refresh in progress" promise/mutex, the first request that needs a refresh (whether from the timer or a 401) initiates it, and any other request that needs a refresh while one is already in flight awaits that same promise rather than starting its own. After the shared refresh resolves, automatically retry any requests that had failed with 401 while waiting.

Does the site type change this?

Long-form data entry applications (insurance, tax, job applications, surveys, anything where a user might spend 10+ minutes on a single screen before submitting) make this critical, the cost of getting it wrong is directly measurable in lost conversions and angry support tickets. Quick, mostly-read interactions (browsing a content site, a simple lookup tool) are far more tolerant of an occasional "please log in again" since there is little unsaved state to lose.